

C++ Qt Vector Drawing Application

Rami Jay Ketanbhai - 2024CS10510

February 14, 2026

1 Installation & Execution

This project uses **CMake** as the build system. Follow these steps to compile and run the application on Unix-like systems (Linux/macOS).

1.1 Prerequisites

- C++17 compliant compiler (GCC/Clang)
- CMake (Version 3.16 or higher)
- Qt 6 (specifically the `Widgets` module)

1.2 Build Instructions

Open a terminal in the project root directory and execute the following commands:

```
# 1. Create a build directory to keep sources clean
mkdir build
cd build

# 2. Generate the Makefiles
cmake -G "Unix_Makefiles" ..

# 3. Compile the project
make

# 4. Run the application
./assign1
```

2 Project Structure

The project is organized into modular folders to separate logic, UI, and utilities.

```
assign1/
├── CMakeLists.txt .....Build configuration
├── resources.qrc .....Resource file for icons
├── src/
│   ├── main.cpp .....Entry point
│   ├── mouseEvents.h .....Custom Scene definition
│   └── mouseEvents*.cpp .....Input handling logic
├── ui/
│   ├── mainwindow.cpp .....Main window logic
│   ├── Renderer.cpp .....View configuration
│   ├── Toolbar.cpp .....UI Toolbar setup
│   └── Menubar.cpp .....UI Menubar setup
└── shapes/
    ├── GraphicsObject.h .....Base class for all shapes
    └── Rectangle, Circle, Line .....Specific shape implementations
```

```

├── utils/
│   ├── SVGparser.cpp .....Save/Load functionality
│   └── icons/ .....SVG/PNG assets for UI
└── resources.qrc .....for icon resources

```

3 Overview: How Objects are Drawn

3.1 The Data Model (MainWindow)

The application state is maintained in `MainWindow` as a vector of smart pointers:

```
std::vector<std::unique_ptr<GraphicsObject>> diagramObjects;
```

This vector owns all shapes (Rectangles, Circles, etc.).

3.2 The Scene (mouseEvents)

The `mouseEvents` class inherits from `QGraphicsScene`. It acts as the Controller:

- **Input Handling:** It captures mouse clicks and drags via `mousePressEvent` and `mouseMoveEvent`.
- **Preview Rendering:** While drawing, it creates a temporary `QGraphicsItem` (e.g., `previewItem`) to show the user what the shape looks like in real-time.
- **Finalization:** Upon mouse release, the shape is finalized, created as a concrete C++ object (e.g., `Rectangle`), and moved into the `diagramObjects` vector.

3.3 The View (QGraphicsView)

Initialized in `ui/Renderer.cpp`, the View is a widget that visualizes the Scene. It handles zooming and scrolling but does not manage the shape data itself.

3.4 The Rendering Loop

The visibility mechanism relies on a synchronization signal:

1. **Modification:** The user draws a shape. The `mouseEvents` class adds the new object to `diagramObjects`.
2. **Signal Emission:** `mouseEvents` emits the `sceneChanged()` signal.
3. **Slot Activation:** `MainWindow` receives this signal and triggers `updateSceneFromModel()`.
4. **Redraw:** This function wipes the visual scene and iterates through `diagramObjects`, calling the `draw()` method of each object to paint it back onto the canvas.